DASH-99: Best Practice Summary

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/26/2026

Overview

This notebook consolidates every actionable best practice from the DASH series (DASH-01 through DASH-07) into a single definitive reference. Each practice specifies exactly what to set, with no ambiguity. Use this as a checklist when building, auditing, or reviewing Dynatrace dashboards.

Table of Contents

1. Dashboard Architecture
2. Design and Layout
3. Tile Configuration
4. Executive Tier
5. Operations Tier
6. Engineering Tier
7. DQL Query Standards
8. Variables and Filters
9. Refresh and Performance
10. Sharing and Permissions
11. Reporting and Automation
12. Dashboard Lifecycle

Prerequisites

Requirement	Details
Prior Reading	DASH-01 through DASH-07 for full context
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`storage:logs:read`, `storage:metrics:read`, `storage:events:read`, `storage:spans:read`, `document:documents:write`

1. Dashboard Architecture

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use the three-tier hierarchy	Create exactly three tiers: Executive, Operations, Engineering	Critical	DASH-02
2	One purpose per dashboard	Each dashboard answers one audience's questions; never build an "everything" dashboard	Critical	DASH-01
3	Connect tiers with drill-down links	Add markdown tiles with dashboard links so users navigate Executive > Operations > Engineering	Recommended	DASH-02
4	Use dashboards for monitoring, notebooks for investigation	Dashboards = continuous monitoring, team visibility, stakeholder reporting. Notebooks = ad-hoc exploration, incident investigation, training	Critical	DASH-01
5	Group related tiles under header tiles	Dashboards have no collapsible section container — place related tiles together under a Markdown header tile (e.g., "Service Health", "Infrastructure")	Recommended	DASH-01

2. Design and Layout

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Place the most critical KPI in the top-left	Top-left tile is the first thing the eye hits; put your primary health indicator there	Critical	DASH-01
7	Follow Z-pattern reading order	Arrange tiles left-to-right, top-to-bottom so the dashboard reads naturally	Recommended	DASH-01
8	Limit tile count to 8-12 per dashboard	More than 15 tiles creates cognitive overload	Critical	DASH-01
9	Use descriptive tile titles	Title format: `Metric Name (Unit) — Scope`. Example: "Error Rate (%) — Checkout Service". Never use "Query 1"	Critical	DASH-01
10	Keep all tiles on the same time range	Use the dashboard time selector for all tiles; only use fixed ranges when there is a specific analytical reason	Recommended	DASH-01
11	Use color intentionally	Green = healthy, Yellow = watch, Red = act. No decorative colors	Recommended	DASH-03
12	No decorative elements	No gratuitous pie charts, no 3D effects, no logos	Optional	DASH-03
13	Executive layout: headline-trend-detail	Top row: 3 single-value KPIs. Middle row: trend line chart. Bottom row: detail table	Recommended	DASH-03

3. Tile Configuration

#	Best Practice	Recommended Setting/Value	Priority	Source
14	Every tile must trigger an action	If a tile does not lead to a decision (investigate, escalate, scale, correlate), remove it	Critical	DASH-01

#	Best Practice	Recommended Setting/Value	Priority	Source
15	Match tile type to data shape	Single value: KPIs. Line chart: trends over time. Bar chart: categorical comparisons. Table: detailed data / top-N. Honeycomb: entity health grid	Recommended	DASH-01
16	Use single-value tiles for executive KPIs	Availability %, MTTR, active problem count, error budget remaining	Critical	DASH-03
17	Use line charts for time trends	Problem count over 7d, error rate over 2h, CPU usage over 1h	Recommended	DASH-02
18	Use table tiles for engineering deep-dives	Engineers need exact values (endpoint p95, DB query time, trace IDs), not visual approximations	Recommended	DASH-05

4. Executive Tier

#	Best Practice	Recommended Setting/Value	Priority	Source
19	Limit to 4-6 tiles maximum	Single-value and trend tiles only	Critical	DASH-02, DASH-03
20	Select KPIs using the SMART filter	Each KPI must be Specific, Measurable, Actionable, Relevant, Time-bound	Critical	DASH-03
21	Start with exactly these 5 KPIs	Availability %, MTTR, Active Problem Count, Problem Trend (7d), Overall Error Rate	Critical	DASH-03
22	Set availability thresholds	Green: >99.9%, Yellow: >99.5%, Red: <99.5%	Critical	DASH-03
23	Set MTTR thresholds	Green: <1 hour, Yellow: <4 hours, Red: >4 hours	Critical	DASH-03
24	Set active problem thresholds	Green: 0, Yellow: 1-3, Red: >3	Recommended	DASH-03

#	Best Practice	Recommended Setting/Value	Priority	Source
25	Set error rate thresholds	Green: <1%, Yellow: <5%, Red: >5%	Recommended	DASH-03
26	Track error budget for SLA targets	99.9% SLA = 43.2 min/month budget. 99.5% = 3.6 hours. 99.0% = 7.2 hours	Recommended	DASH-03
27	Use 7d-30d time ranges	Executive dashboards show weekly or monthly trends, not minute-by-minute data	Critical	DASH-02
28	No technical jargon	"Availability" not "HTTP 200 ratio". "Resolution Time" not "MTTR" if audience is non-technical	Recommended	DASH-02
29	Exclude duplicate detected problems	Always filter `dt.davis.is_duplicate == false`. Do not add `dt.davis.is_frequent_event` — frequent issue detection is being phased out and the flag no longer separates anything (ADOPT-03 §5)	Critical	DASH-03
30	Dashboard must be understandable in <30 seconds	If it takes longer, push detail down to operations tier	Critical	DASH-02

5. Operations Tier

#	Best Practice	Recommended Setting/Value	Priority	Source
31	Use 8-12 tiles with mixed chart types	Line charts for trends, bar charts for comparisons, tables for top-N, single values for status	Critical	DASH-02, DASH-04
32	Use 1h-2h time windows for real-time tiles	Short windows surface real-time spikes; 24h hides them	Critical	DASH-02, DASH-04

#	Best Practice	Recommended Setting/Value	Priority	Source
33	Show p50, p90, p95 response time lines	Three percentile lines on a single timeseries chart at 5-minute intervals	Recommended	DASH-04
34	Include active problems table	Show display_id, event.name, event.category, duration_min. Sort by duration descending	Critical	DASH-04
35	Track log volume by severity level	Stacked area chart: `makeTimeseries count(), interval:5m, by:{loglevel}`. Filter out loglevel == "NONE"	Recommended	DASH-04
36	Detect log volume spikes	Compare current hour log count to same hour yesterday using `append`	Recommended	DASH-04
37	Include deployment event table	Filter `event.type == "CUSTOM_DEPLOYMENT" or (event.kind == "SDLC_EVENT" and event.category == "task" and event.type == "deployment")` — classic and SDLC deployment events. Show last 6h	Critical	DASH-04
38	Add infrastructure tiles for CPU and memory	`timeseries avg(dt.host.cpu.usage)` and `timeseries avg(dt.host.memory.usage)` by host	Recommended	DASH-04
39	Show concurrently open problems over time	Use `spread:timeframe(from:event.start, to:coalesce(event.end, now()))` in makeTimeseries	Recommended	DASH-04
40	Avoid spaghetti line charts	Use top-N filtering or variables instead of plotting all services on one chart	Critical	DASH-04

6. Engineering Tier

#	Best Practice	Recommended Setting/Value	Priority	Source
41	Use 10-15 tiles with high data density	Tables, detailed charts, multiple variables	Recommended	DASH-02, DASH-05
42	Use 15m-1h time ranges for investigation	Longer ranges dilute the signal; shorter ranges focus on the incident window	Critical	DASH-05
43	Include service throughput table	Show request_count, avg_ms, p50_ms, p95_ms, p99_ms, error_rate_pct per service	Critical	DASH-05
44	Include endpoint performance table	Show requests, avg_ms, p95_ms, error_rate by http.route and http.request.method	Critical	DASH-05
45	Include database performance table	Show avg_ms, p95_ms, query_count, error_rate by db.system and db.namespace — statements only: exclude `CONNECT`, `COMMIT`, `PREPARE`, `RESULTSET` (`db.operation.name`)	Recommended	DASH-05
46	Always filter `isNotNull(db.system)` on DB spans	Prevents null groups in database analysis	Critical	DASH-05
47	Include latency distribution histogram	Bucket spans into <100ms, 100-500ms, 500ms-1s, 1s-5s, >5s categories	Recommended	DASH-05
48	Provide before/after deployment comparison	Use `append` to compare 2h after vs 2h before for avg_ms, p95_ms, error_rate	Recommended	DASH-05
49	Include trace.id in error span tables	Engineers need clickable trace IDs for distributed trace investigation	Critical	DASH-05

#	Best Practice	Recommended Setting/Value	Priority	Source
50	Add markdown tiles documenting each section	Explain what the section shows and how to interpret it	Recommended	DASH-05
51	Use variables extensively	Service and namespace variables (DQL or List) so engineers can focus on their area; time comes from the dashboard timeframe selector	Critical	DASH-05

7. DQL Query Standards

#	Best Practice	Recommended Setting/Value	Priority	Source
52	Always specify a time range on fetch	`fetch logs, from:-1h`. Never use bare `fetch logs`	Critical	DASH-01, all
53	Filter before aggregating	Place `filter` immediately after `fetch`, never after `sort` or `summarize`	Critical	DASH-01
54	Alias all aggregation results	`summarize c = count()` not `summarize count()`. Required for sort/fieldsAdd references	Critical	All
55	Convert span duration to milliseconds	`duration / 1ms` for display. Duration is stored in nanoseconds	Critical	DASH-04, DASH-05
56	Convert detected problem duration to hours	`resolved_problem_duration / 1h`. Duration is in nanoseconds	Critical	DASH-03
57	Use `arrayAvg()` on timeseries results before filter/sort	Timeseries returns arrays, not scalars. Use `fieldsAdd val = arrayAvg(ts)` then `filter val > X`	Critical	DASH-04

#	Best Practice	Recommended Setting/Value	Priority	Source
58	Use `countIf()` for conditional counts	`errors = countIf(span.status_code == "error")` instead of separate filter + count	Recommended	DASH-03
59	Calculate error rate as percentage	`100.0 * errors / total`. Always multiply by 100.0 (float), not 100 (int)	Recommended	DASH-04
60	Filter out duplicate detected problems	Always include `filter dt.davis.is_duplicate == false` in problem queries	Critical	DASH-03
61	Use `round()` with named `decimals:` parameter	`round(value, decimals: 2)` not `round(value, 2)`	Critical	All
62	Prototype all queries in a notebook first	Validate DQL in a notebook before transferring to dashboard tiles	Recommended	DASH-01
63	Use `span.status_code` for span failure — never `otel.status_code`	`otel.status_code` and `otel.status_message` **do not exist** (no row in `dt.semantic_dictionary.fields`). The real fields are `span.status_code` (`stable`) and `span.status_message` (`experimental`)	Critical	DASH-03, DASH-04, DASH-05
64	Match `span.status_code` in **lowercase**	`== "error"`, never `== "ERROR"`. The OTel SDK constant is uppercase; the Grail value is not. On a live tenant `"ERROR"` matched 0 spans against 18,630 for `"error"` (08/11/2026)	Critical	DASH-03, DASH-04, DASH-05
65	Derive span successes as `total - errors`	`span.status_code` is **null on successful spans** — `"ok"` is written vanishingly rarely — so `countIf(span.status_code != "error")` counts almost nothing and an availability tile reads ~0% on a healthy estate	Critical	DASH-03

> **Rules 63–65 are one bug, caught three ways.** Each of the three mistakes returns a *plausible number* rather than an error, so nothing in the tile, the query, or the dashboard tells you it is wrong. The order in which they bite matters: fixing only the field name (63) turns a hard failure into a silent zero (64), and fixing the name and the case but not the null-handling (65) turns a correct-looking rename into a **0% availability** executive tile on a service that is actually running at 99.6%. Verified end to end against a live tenant, 08/11/2026.

8. Variables and Filters

#	Best Practice	Recommended Setting/Value	Priority	Source
66	Use DQL variables for services and hosts	Variable type: DQL, with a query over `smartscapeNodes` or `dt.entity.service` / `dt.entity.host`. The Dashboards app has no entity-selector type	Critical	DASH-06
67	Use List variables for fixed sets: namespaces, log levels, environments	Values discovered from actual data via DQL; switch to a DQL variable when the set changes often	Recommended	DASH-06
68	Reference the variable key, not its display name	DQL uses the key, derived from the name with every non-alphanumeric character replaced by `_`. Keep keys lowercase with underscores: `$k8s_namespace`, `$service`, `$environment`	Critical	DASH-06
69	Reference variables in filter clauses	`filter dt.entity.host == $host` for DQL entity variables; `filter k8s.namespace.name == $namespace` for List or Free-text; `filter in(loglevel, array($log_levels))` for multi-select Never wrap `$var` in quotes — the substitution adds them	Critical	DASH-06

#	Best Practice	Recommended Setting/Value	Priority	Source
70	Provide an "All" option on every variable	Let users remove the filter to see aggregate data	Recommended	DASH-06
71	Use the same variable name across all related tiles	Ensures filter propagation: changing the dropdown updates every tile that references it	Critical	DASH-06
72	Chain variables for hierarchical filtering	Second variable's query-based options reference the first variable (e.g., cluster > namespace)	Optional	DASH-06
73	Build Golden Signals template dashboards	One variable-driven template with Latency, Traffic, Errors, Saturation sections. Works for any service	Recommended	DASH-06
74	Test variables with extreme values	Select the busiest and quietest entities to verify the layout renders correctly under load	Recommended	DASH-06

9. Refresh and Performance

#	Best Practice	Recommended Setting/Value	Priority	Source
75	Executive wall screen refresh	Set to 10-15 minutes	Critical	DASH-02
76	Operations NOC wall refresh	Set to 1-2 minutes	Critical	DASH-02
77	Operations on-call refresh	Set to 2-5 minutes	Recommended	DASH-02
78	Engineering investigation refresh	Set to manual (on-demand)	Recommended	DASH-02
79	Executive weekly review refresh	Set to manual	Optional	DASH-02

#	Best Practice	Recommended Setting/Value	Priority	Source
80	Never set refresh below 1 minute	Aggressive refresh on complex queries causes unnecessary load. Start at 5 min, decrease only if critical	Critical	DASH-02
81	Use `limit` on all top-N queries	Always append `| limit N` (10-20) to prevent unbounded result sets in tiles	Critical	DASH-04, DASH-05
82	Use `filterOut` instead of `not` for negation filters	`filterOut loglevel == "NONE"` is faster than `filter not loglevel == "NONE"`	Recommended	DASH-04

10. Sharing and Permissions

#	Best Practice	Recommended Setting/Value	Priority	Source
83	Executive dashboards: platform team owns, leadership views	Owner: platform team lead. Editors: platform team. Viewers: all managers + leadership	Critical	DASH-07
84	Operations dashboards: SRE team owns	Owner: SRE team. Editors: SRE + on-call. Viewers: operations group	Critical	DASH-07
85	Engineering dashboards: service team owns	Owner: service team lead. Editors: service team. Viewers: engineering org	Recommended	DASH-07
86	Limit editor access	Encourage clone-and-customize over editing shared originals	Critical	DASH-07
87	Use naming convention for all dashboards	Format: `[Tier] - [Team/Service] - [Purpose]`. Example: "Ops - Checkout - Service Health"	Critical	DASH-07

#	Best Practice	Recommended Setting/Value	Priority	Source
88	Tag dashboards with tier, team, and service	Enables filtering and discovery in the dashboard library	Recommended	DASH-07
89	Maintain a dashboard ownership registry	Document who owns each dashboard for accountability	Recommended	DASH-07
90	Archive dashboards not used in 90 days	Proxy: no tile queries from the Dashboards app in 90 days (`QUERY_EXECUTION_EVENT` in `dt.system.events`) — document reads are not audited. Regular cleanup prevents library bloat	Recommended	DASH-07

11. Reporting and Automation

#	Best Practice	Recommended Setting/Value	Priority	Source
91	Automate executive summary via Workflow	Schedule: weekly, Monday 8 AM. Recipients: leadership, management	Recommended	DASH-07
92	Automate operations daily report via Workflow	Schedule: daily, 7 AM. Recipients: SRE team, on-call	Recommended	DASH-07
93	Automate SLA compliance report via Workflow	Schedule: monthly, 1st of month. Recipients: account management, leadership	Recommended	DASH-07

#	Best Practice	Recommended Setting/Value	Priority	Source
94	Automate cost/volume tracking report	Schedule: weekly. Recipients: platform team, finance	Optional	DASH-07
95	Only report services with error rate >1% in daily ops report	Filter `error_rate_pct > 1.0` to surface only degraded services	Recommended	DASH-07
96	Export dashboard JSON for backup	Use Documents API: `GET /platform/document/v1/documents/<id>` returns metadata and content as one multipart response — store the content part in version control	Recommended	DASH-07

12. Dashboard Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Source
97	Follow the 6-step creation workflow	1) Define audience/purpose, 2) Identify 5-10 key metrics, 3) Prototype in notebook, 4) Build dashboard, 5) Add variables, 6) Share and iterate	Critical	DASH-01
98	Manage dashboards as code	Use Monaco CLI or Terraform for multi-environment deployment, version control, and peer review	Recommended	DASH-07

#	Best Practice	Recommended Setting/Value	Priority	Source
99	Validate every dashboard payload before merging it	Deploy to a non-production tenant, open the dashboard in the Dashboards app, confirm **zero** validation warnings. API acceptance (`2xx`) is not render confirmation. Since SaaS 1.344 (stricter rules in SaaS 1.346) a dashboard that fails validation does not load at all — confirm which version your tenant runs — and API- or AI-authored dashboards are the named affected population	Critical	DASH-07
100	Organize dashboard repo by tier	Structure: `dashboards/executive/` , `dashboards/operations/` , `dashboards/engineering/` , `dashboards/templates/`	Recommended	DASH-07
101	Deploy dashboard changes through PR review	Export > modify in feature branch > PR review > deploy to staging > validate schema/render > review data fidelity > deploy to production	Recommended	DASH-07
102	Maintain a query registry	Track which DQL queries power which dashboard tiles. Update when DQL syntax or data sources change	Optional	DASH-07
103	A dashboard is never "done"	Share with target audience, collect feedback, and refine continuously. Evolve with team needs	Recommended	DASH-01
104	Validate dashboard data matches manual reports	Executives lose trust if dashboard numbers differ from status meeting reports. Cross-check availability and MTTR calculations	Critical	DASH-03

Summary

This notebook contains **104 best practices** extracted from the DASH series, organized into 12 categories:

Category	Practices	Critical Count
Dashboard Architecture	1-5	3
Design and Layout	6-13	3
Tile Configuration	14-18	2
Executive Tier	19-30	8
Operations Tier	31-40	5
Engineering Tier	41-51	6
DQL Query Standards	52-65	11
Variables and Filters	66-74	4
Refresh and Performance	75-82	4
Sharing and Permissions	83-90	4
Reporting and Automation	91-96	0
Dashboard Lifecycle	97-104	3

Priority breakdown: 53 Critical, 46 Recommended, 5 Optional. Start with Critical practices first, then layer in Recommended practices for a mature dashboard strategy.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.